

中国人民大学 2022-2023 学年第 2 学期《机器学习基础》 期末试题（A 卷）个人答案

JEEdward

2025 年 6 月 4 日

判断题

参考答案：

小题	答案
1	是
2	是
3	否
4	否

不定项选择题

参考答案：

小题	答案
1	AC
2	ABC
3	BD
4	BCD

算法设计题 1

Answer(1)

1. 从 $\text{Uniform}(0, 1)$ 样本生成器中生成一个随机数 U 。
2. 如果 $U < 0.5$ ，则输出 0。
3. 否则（如果 $U \geq 0.5$ ），则输出 1。

Python 代码示例

```
1 import random
2
3 def bernoulli_0_5_sampler():
4     """
```

```

5  设计一个伯努利分布 Bernoulli(0.5) 的样本生成器
6  基于 Uniform(0,1) 样本生成器
7  """
8  u = random.random() # 生成一个 Uniform(0,1) 的随机数
9  if u < 0.5:
10     return 0
11  else:
12     return 1
13
14 # 测试
15 print("--- 伯努利分布 Bernoulli(0.5) 样本生成器 ---")
16 samples_bernoulli = [bernoulli_0_5_sampler() for _ in range(10)]
17 print(f"生成的伯努利样本: {samples_bernoulli}")
18
19 # 简单统计验证
20 counts = {0: samples_bernoulli.count(0), 1: samples_bernoulli.count(1)}
21 print(f"样本统计: {counts}")

```

Listing 1: 伯努利分布 Bernoulli(0.5) 样本生成器

Answer(2)

关键方法：逆变换采样 (Inverse Transform Sampling)。

1. 首先找到指数分布的累积分布函数 (CDF) $F(x)$ 。

$$F(x) = \int_0^x e^{-t} dt = -e^{-t} \Big|_0^x = -e^{-x} - (-e^0) = 1 - e^{-x}, \quad x \geq 0$$

2. 令 $U = F(x)$ ，其中 $U \sim \text{Uniform}(0, 1)$ 。

$$U = 1 - e^{-x}$$

3. 解出 x :

$$e^{-x} = 1 - U$$

$$x = -\ln(1 - U)$$

注意：因为 $U \sim \text{Uniform}(0, 1)$ ，所以 $1 - U$ 也服从 $\text{Uniform}(0, 1)$ 分布。因此，我们也可以直接使用 $X = -\ln(U)$ 。

Python 代码示例

```

1  import random
2  import math
3
4  def exponential_sampler():
5      """
6      设计一个指数分布 p(X) = exp(-X), X >= 0 的样本生成器
7      基于 Uniform(0,1) 样本生成器
8      """
9      u = random.random() # 生成一个 Uniform(0,1) 的随机数
10     x = -math.log(1 - u) # 或者 -math.log(u)
11     return x

```

```

12
13 # 测试
14 print("\n--- 指数分布  $p(X) = \exp(-X)$  样本生成器 ---")
15 samples_exp = [exponential_sampler() for _ in range(10)]
16 print(f"生成的指数样本: {samples_exp}")
17
18 # 简单验证均值 (理论均值为1)
19 if samples_exp: # Ensure list is not empty
20     print(f"样本均值: {sum(samples_exp) / len(samples_exp):.2f}")

```

Listing 2: 指数分布 $p(X) = \exp(-X)$ 样本生成器

Answer(3)

当尺度矩阵是 $\sigma \mathbf{I}_D$ 时, 多元拉普拉斯分布可以分解为 D 个独立同分布的标量拉普拉斯随机变量。每个分量 x_i 服从一个中心化在 b_i 且尺度为 σ 的拉普拉斯分布。其概率密度函数为:

$$p(x_i) = \frac{1}{2\sigma} \exp\left(-\frac{|x_i - b_i|}{\sigma}\right)$$

一个标准拉普拉斯分布 (均值 0, 尺度 1) 可以通过以下方式从均匀分布生成 (利用逆 CDF 方法), 可以参考 Wikipedia:

设 $U \sim \text{Uniform}(0, 1)$ 。

- 如果 $U \leq 0.5$, 则 $X = \ln(2U)$ 。
- 如果 $U > 0.5$, 则 $X = -\ln(2(1 - U))$ 。

对于尺度为 σ 、均值为 0 的拉普拉斯分布 $p(X) = \frac{1}{2\sigma} e^{-|X|/\sigma}$:

- 如果 $U \leq 0.5$, 则 $X = \sigma \ln(2U)$ 。
- 如果 $U > 0.5$, 则 $X = -\sigma \ln(2(1 - U))$ 。

于是, 对于每个维度 d 从 1 到 D :

- 从 $\text{Uniform}(0, 1)$ 样本生成器中生成一个随机数 U_d 。
- 根据 U_d 和给定的 σ 生成一个均值为 0、尺度为 σ 的拉普拉斯样本 X_d^{std} :
 - 如果 $U_d < 0.5$, 则 $X_d^{\text{std}} = \sigma \cdot \ln(2U_d)$ 。
 - 否则 (如果 $U_d \geq 0.5$), 则 $X_d^{\text{std}} = -\sigma \cdot \ln(2(1 - U_d))$ 。
- 将位置参数 b_d 添加到 X_d^{std} , 得到最终的样本分量 $x_d = X_d^{\text{std}} + b_d$ 。

将所有分量 $x_1, x_2, \dots, x_D$ 组合成 D 维向量 $\mathbf{x}$ 。输出 $\mathbf{x}$ 。

Python 代码示例

```

1 import random
2 import math
3 import numpy as np
4
5 def laplace_sampler_scalar(sigma):
6     """

```

```

7  生成一个均值0, 尺度sigma的标量拉普拉斯随机变量
8  基于 Uniform(0,1) 样本生成器
9  """
10 u = random.random()
11 if u < 0.5:
12     # For u -> 0, log(2u) -> -inf. This is correct.
13     return sigma * math.log(2 * u)
14 else:
15     # For u -> 1, log(2(1-u)) -> -inf. This is correct.
16     return -sigma * math.log(2 * (1 - u))
17
18 def laplace_multivariate_sampler(b, sigma):
19     """
20     设计一个 D 维 Laplace 分布 Laplace(b, sigma * I_D) 的样本生成器
21     基于 Uniform(0,1) 样本生成器和上述标量拉普拉斯生成器
22     b: 位置向量 (list or numpy array)
23     sigma: 尺度参数 (float)
24     """
25     D = len(b)
26     x = np.zeros(D)
27     for i in range(D):
28         x[i] = laplace_sampler_scalar(sigma) + b[i]
29     return x
30
31 # 测试
32 print("\n--- D 维 Laplace 分布 Laplace(b, sigma * I_D) 样本生成器 ---")
33 D_dim = 3
34 b_vector = np.array([1.0, 2.0, 0.5])
35 sigma_val = 1.0 # Example scale
36
37 samples_laplace = [laplace_multivariate_sampler(b_vector, sigma_val) for _ in range(5)]
38 print(f"D={D_dim}, b={b_vector}, sigma={sigma_val}")
39 print(f"生成的多元拉普拉斯样本:")
40 for sample in samples_laplace:
41     print(sample)
42
43 # 简单验证均值 (应该接近b)
44 if samples_laplace: # Ensure list is not empty
45     mean_samples = np.mean(samples_laplace, axis=0)
46     print(f"样本均值 (应接近b): {mean_samples}")

```

Listing 3: D 维 Laplace 分布样本生成器

算法设计题 2

MAP 的目标是找到使后验概率 $p(\mathbf{w}|\mathbf{x}, y)$ 最大的 $\mathbf{w}$: $\mathbf{w}_{\text{MAP}} = \arg \max_{\mathbf{w}} p(\mathbf{w}|\mathbf{x}, y)$
 根据贝叶斯定理:

$$p(\mathbf{w}|\mathbf{x}, y) \propto p(y|\mathbf{x}, \mathbf{w})p(\mathbf{w})$$

$$\log p(\mathbf{w}|\mathbf{x}, y) = \log p(y|\mathbf{x}, \mathbf{w}) + \log p(\mathbf{w}) + \text{常数}$$

因此, 我们需要最大化的目标函数是 $\mathcal{L}(\mathbf{w}) = \log p(y|\mathbf{x}, \mathbf{w}) + \log p(\mathbf{w})$ 。

Answer(1)

似然项的对数：

$$\begin{aligned}\log p(y|\mathbf{x}, \mathbf{w}) &= \log \left(\frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y - \mathbf{x}^T \mathbf{w})^2}{2\sigma^2} \right) \right) \\ &= \log \left(\frac{1}{\sqrt{2\pi}\sigma} \right) - \frac{(y - \mathbf{x}^T \mathbf{w})^2}{2\sigma^2}\end{aligned}$$

由于 $\log \left(\frac{1}{\sqrt{2\pi}\sigma} \right)$ 是与 $\mathbf{w}$ 无关的常数，在最大化时可以忽略。

先验项的对数：

$$\log p(\mathbf{w}) = \log \left(c \cdot \frac{1}{2^D} \exp(-\|\mathbf{w}\|_1) + (1-c) \cdot \frac{1}{(2\pi)^{D/2}} \exp \left(-\frac{1}{2} \|\mathbf{w}\|_2^2 \right) \right)$$

综合目标函数 (最大化形式)：我们要求解：

$$\mathbf{w}_{\text{MAP}} = \arg \max_{\mathbf{w}} \left(-\frac{(y - \mathbf{x}^T \mathbf{w})^2}{2\sigma^2} + \log \left(c \cdot \frac{1}{2^D} \exp(-\|\mathbf{w}\|_1) + (1-c) \cdot \frac{1}{(2\pi)^{D/2}} \exp \left(-\frac{1}{2} \|\mathbf{w}\|_2^2 \right) \right) \right)$$

为了将其转化为最小化问题（这是优化算法通常的目标形式），我们对其取负号：

$$J(\mathbf{w}) = \frac{(y - \mathbf{x}^T \mathbf{w})^2}{2\sigma^2} - \log \left(c \cdot \frac{1}{2^D} \exp(-\|\mathbf{w}\|_1) + (1-c) \cdot \frac{1}{(2\pi)^{D/2}} \exp \left(-\frac{1}{2} \|\mathbf{w}\|_2^2 \right) \right)$$

这个目标函数是非凸的，主要因为先验项中的对数和的形式以及 $\|\mathbf{w}\|_1$ 的非光滑性。

Answer(2)

由于目标函数 $J(\mathbf{w})$ 的非凸性和 $\|\mathbf{w}\|_1$ 的不可微性，我们可以考虑使用 EM 算法。EM 算法可以处理这个问题先验分布中的混合成分。

EM 算法的动机：先验分布 $p(\mathbf{w})$ 是一个混合分布。我们可以引入一个隐变量 Z ，它指示 $\mathbf{w}$ 是来自混合中的第一个分量（拉普拉斯分布，设 $Z = 1$ ）还是第二个分量（高斯分布，设 $Z = 0$ ）。如果这个隐变量是已知的，那么优化问题就会大大简化。EM 算法通过迭代地估计隐变量的后验（E 步）和优化参数（M 步）来处理这个问题。

算法步骤：

1. 初始化：

- 随机初始化参数 $\mathbf{w}^{(0)}$ 。
- 设定收敛阈值 δ 和最大迭代次数。

2. 主迭代循环 (第 t 次迭代)：

(a) Expectation-Step: 计算隐变量 Z 的后验概率

在当前参数估计 $\mathbf{w}^{(t)}$ 下，我们计算 $\mathbf{w}$ 来自拉普拉斯分量的后验概率 $P(Z = 1|\mathbf{w}^{(t)})$ ，也就是 responsibility。

令 $p_1(\mathbf{w}) = \frac{1}{2^D} \exp(-\|\mathbf{w}\|_1)$ （拉普拉斯分量）

令 $p_2(\mathbf{w}) = \frac{1}{(2\pi)^{D/2}} \exp \left(-\frac{1}{2} \|\mathbf{w}\|_2^2 \right)$ （高斯分量）

根据条件概率公式：

$$P(Z = 1|\mathbf{w}^{(t)}) = \frac{p(\mathbf{w}^{(t)}|Z = 1)P(Z = 1)}{p(\mathbf{w}^{(t)})} = \frac{p_1(\mathbf{w}^{(t)}) \cdot c}{c \cdot p_1(\mathbf{w}^{(t)}) + (1 - c) \cdot p_2(\mathbf{w}^{(t)})}$$

同样，

$$P(Z = 0|\mathbf{w}^{(t)}) = \frac{p_2(\mathbf{w}^{(t)}) \cdot (1 - c)}{c \cdot p_1(\mathbf{w}^{(t)}) + (1 - c) \cdot p_2(\mathbf{w}^{(t)})} = 1 - P(Z = 1|\mathbf{w}^{(t)})$$

不妨设 $P_0 = P(Z = 0|\mathbf{w}^{(t)})$, $P_1 = P(Z = 1|\mathbf{w}^{(t)})$ ，方便 M 步中的描述。

- (b) **Maximization-Step (M-步)：更新参数 $\mathbf{w}$ 。** M 步的目标是最大化完整数据对数后验关于隐变量后验概率的期望，即最大化 $Q(\mathbf{w}|\mathbf{w}^{(t)}) = E_{Z|\mathbf{w}^{(t)}}[\log p(y, Z, \mathbf{w})]$ 。 $Q(\mathbf{w}|\mathbf{w}^{(t)}) = \log p(y|\mathbf{x}, \mathbf{w}) + P_1 \log(p_L(\mathbf{w}) \cdot c) + P_0 \log(p_G(\mathbf{w}) \cdot (1 - c))$ 。忽略与 $\mathbf{w}$ 无关的常数项 (如 $P_1 \log c$, $P_1(-D \log 2)$, $P_0 \log(1 - c)$, $P_0(-\frac{D}{2} \log(2\pi))$)，我们需要最小化以下目标函数：

$$J_M(\mathbf{w}) = \frac{(y - \mathbf{x}^T \mathbf{w})^2}{2\sigma^2} + P_1 \|\mathbf{w}\|_1 + P_0 \frac{1}{2} \|\mathbf{w}\|_2^2$$

这是一个标准的 **Elastic Net** 形式的问题，其 L1 和 L2 惩罚项的权重是动态的，不能直接使用传统的 Gradient Descent / SGD，而需要使用非光滑凸优化的算法。一种可行的做法是迭代软阈值法 (**Iterative Soft-Thresholding Algorithm, ISTA**)。

将目标函数 $J_M(\mathbf{w})$ 分解为两部分：

- 可微部分： $f(\mathbf{w}) = \frac{(y - \mathbf{x}^T \mathbf{w})^2}{2\sigma^2} + P_0 \frac{1}{2} \|\mathbf{w}\|_2^2$ ，其梯度为： $\nabla f(\mathbf{w}) = -\frac{1}{\sigma^2}(y - \mathbf{x}^T \mathbf{w})\mathbf{x} + P_0 \mathbf{w}$
- 不可微部分： $g(\mathbf{w}) = P_1 \|\mathbf{w}\|_1$

设内层迭代索引为 k ，

- (a) 计算光滑部分 $f(\mathbf{w})$ 在当前点 $\mathbf{w}^{(k)}$ 的梯度：

$$\nabla f(\mathbf{w}^{(k)}) = -\frac{1}{\sigma^2}(y - \mathbf{x}^T \mathbf{w}^{(k)})\mathbf{x} + P_0 \mathbf{w}^{(k)}$$

- (b) 执行一步梯度下降得到中间变量 $\mathbf{u}^{(k)}$ ：

$$\mathbf{u}^{(k)} = \mathbf{w}^{(k)} - \eta \nabla f(\mathbf{w}^{(k)})$$

其中 η 为学习率。

- (c) 对 $\mathbf{u}^{(k)}$ 的每个分量应用软阈值操作来获得 $\mathbf{w}^{(k+1)}$ ：对于 $j = 1, \dots, D$ ：

$$w_j^{(k+1)} = \text{sgn}(u_j^{(k)}) \cdot \max(0, |u_j^{(k)}| - \eta P_1)$$

这可以记为 $\mathbf{w}^{(k+1)} = S_{\eta P_1}(\mathbf{u}^{(k)})$ ，其中 $S_{\lambda'}(\cdot)$ 是阈值为 λ' 的软阈值算子，其定义如 Lecture 4 (Slide 17, Eq 15) 所示： $S_{\lambda'}(a) = \text{sgn}(a) \max(0, |a| - \lambda')$ 。

这个内层优化循环 (a-c) 需要运行多次迭代，直到 $\mathbf{w}$ 在 M 步中收敛到 $J_M(\mathbf{w})$ 的一个近似最优解，该解为 $\mathbf{w}^{(t+1)}$ 。

3. **检查收敛**：如果 $\|\mathbf{w}^{(t+1)} - \mathbf{w}^{(t)}\| < \delta$ 或达到最大迭代次数，则停止迭代。否则，重复主迭代循环。

证明题 1

证明 (1): 对于局部线性嵌入 (LLE) 和 Laplace 特征映射 (LE), 它们构造的矩阵 Φ 满足 $\Phi \mathbf{1}_N = \mathbf{0}_N$ 。

要证明 $\Phi \mathbf{1}_N = \mathbf{0}_N$, 我们需要理解 LLE 和 LE 中 Φ 矩阵的构造原理, 并找到它们与全一向量 $\mathbf{1}_N$ 的关系。

对于 LLE: LLE 的 Φ 矩阵定义为:

$$\Phi = \mathbf{I} - \tilde{\mathbf{W}} - \tilde{\mathbf{W}}^T + \tilde{\mathbf{W}}^T \tilde{\mathbf{W}}$$

其中 $\tilde{\mathbf{W}}$ 是一个 $N \times N$ 的矩阵, 其元素 $\tilde{w}_{ij}$ 在 $\mathbf{x}_j$ 是 $\mathbf{x}_i$ 的邻居时取 LLE 重构权重 w_{ij} , 否则为 0。

LLE 在计算重构权重 w_{ij} 时, 有一个重要的约束条件是每个点的权重和为 1, 即 $\sum_{k=1}^K w_{ik} = 1$ (对于 $\mathbf{x}_i$ 及其邻居 $\mathbf{x}_k$), 这意味着 $\tilde{\mathbf{W}}$ 每一行的非零元素之和为 1。因此 $\tilde{\mathbf{W}} \mathbf{1}_N = \mathbf{1}_N$, 于是:

$$\begin{aligned} \Phi \mathbf{1}_N &= (\mathbf{I} - \tilde{\mathbf{W}} - \tilde{\mathbf{W}}^T + \tilde{\mathbf{W}}^T \tilde{\mathbf{W}}) \mathbf{1}_N \\ &= \mathbf{I} \mathbf{1}_N - \tilde{\mathbf{W}} \mathbf{1}_N - \tilde{\mathbf{W}}^T \mathbf{1}_N + \tilde{\mathbf{W}}^T (\tilde{\mathbf{W}} \mathbf{1}_N) \\ &= \mathbf{1}_N - \mathbf{1}_N - \tilde{\mathbf{W}}^T \mathbf{1}_N + \tilde{\mathbf{W}}^T \mathbf{1}_N \\ &= \mathbf{0}_N \end{aligned}$$

对于 LE: 在 LE 中, Φ 矩阵是图拉普拉斯矩阵 $\mathbf{L} = \text{diag}(\mathbf{A} \mathbf{1}_N) - \mathbf{A}$, 其中 $\mathbf{A}$ 是相似度矩阵。于是:

$$\mathbf{L} \mathbf{1}_N = \text{diag}(\mathbf{A} \mathbf{1}_N) \mathbf{1}_N - \mathbf{A} \mathbf{1}_N$$

对于 $\text{diag}(\mathbf{A} \mathbf{1}_N) \mathbf{1}_N$, 由于 $\text{diag}(\mathbf{A} \mathbf{1}_N)$ 是对角矩阵, 其第 i 个对角线元素是 $\sum_j A_{ij}$, 所以 $\text{diag}(\mathbf{A} \mathbf{1}_N) \mathbf{1}_N$ 的第 i 个元素是 $\sum_j A_{ij} \cdot 1 = \sum_j A_{ij}$ 。对于 $\mathbf{A} \mathbf{1}_N$, 其第 i 个元素是 $\sum_j A_{ij} \cdot 1 = \sum_j A_{ij}$ 。因此, $\mathbf{L} \mathbf{1}_N = \mathbf{0}_N$ 。

证明 (2): 证明当 Φ 是正定矩阵的情况下 $\mathbf{Z} = \mathbf{U}_L$, 其中 $\mathbf{U}_L = [\mathbf{u}_1, \dots, \mathbf{u}_L]$ 表示最小的 L 个特征值对应的特征向量构成的矩阵。

这个问题的核心是求解一个带有正交约束的二次型最小化问题, 这种问题通常通过拉格朗日乘子法转化为特征值分解问题。

1. 构造拉格朗日函数:

$$\mathcal{L}(\mathbf{Z}, \mathbf{\Lambda}) = \text{trace}(\mathbf{Z}^T \Phi \mathbf{Z}) - \text{trace}(\mathbf{\Lambda}(\mathbf{Z}^T \mathbf{Z} - \mathbf{I}_L))$$

其中 $\mathbf{\Lambda}$ 是一个 $L \times L$ 的拉格朗日乘子矩阵。这里我们假设 $\mathbf{\Lambda}$ 是一个对称矩阵。

对 $\mathcal{L}(\mathbf{Z}, \mathbf{\Lambda})$ 关于 $\mathbf{Z}$ 求导, 并令导数等于零:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}} = 2\Phi \mathbf{Z} - 2\mathbf{Z} \mathbf{\Lambda} = \mathbf{0}$$

$$\Phi \mathbf{Z} = \mathbf{Z} \mathbf{\Lambda}$$

如果我们将 $\mathbf{Z}$ 看作是 L 个列向量 $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_L]$, 那么方程可以写成:

$$\Phi \mathbf{z}_j = \lambda_j \mathbf{z}_j, \quad \text{for } j = 1, \dots, L$$

这意味着 $\mathbf{z}_j$ 是矩阵 Φ 的特征向量， λ_j 是对应的特征值。又注意到约束条件 $\mathbf{Z}^T \mathbf{Z} = \mathbf{I}_L$ ，这表示 $\mathbf{z}_j$ 都是正交的单位向量（即 $\mathbf{z}_j^T \mathbf{z}_k = \delta_{jk}$ ）。

将 $\Phi \mathbf{Z} = \mathbf{Z} \Lambda$ 代入目标函数：

$$\text{trace}(\mathbf{Z}^T \Phi \mathbf{Z}) = \text{trace}(\mathbf{Z}^T \mathbf{Z} \Lambda) = \text{trace}(\mathbf{I}_L \Lambda) = \text{trace}(\Lambda)$$

由于 Λ 是对角矩阵，其对角线元素就是特征值 $\lambda_1, \dots, \lambda_L$ 。因此，目标函数等于 $\sum_{j=1}^L \lambda_j$ 。于是最优的 $\mathbf{Z}$ 应该由 Φ 的最小的 L 个特征值对应的特征向量组成。而 Φ 是正定矩阵，其特征值都是正数。这一点很关键，这意味着它没有零特征值。因此，我们直接选取最小的 L 个特征值即可。

证明题 2

警醒：现阶段笔者没有能力解决这个问题，尤其对于后来者，不建议作答这道题目，不会对你巩固知识点有过多的好处！以下内容纯由 LLM 生成：

Answer(1)

令向量

$$\mathbf{m}^{(t)} = \begin{pmatrix} m^{(t)}(x_1) \\ m^{(t)}(x_2) \\ \vdots \\ m^{(t)}(x_N) \end{pmatrix} \in \mathbb{R}^{N \times D},$$

其中每个 $m^{(t)}(x_n) \in \mathbb{R}^D$ 。为说明收敛性，我们分解到各个维度上：对第 k 维，用

$$\mathbf{m}_{(k)}^{(t)} = (m_{(k)}^{(t)}(x_1), m_{(k)}^{(t)}(x_2), \dots, m_{(k)}^{(t)}(x_N))^T \in \mathbb{R}^N,$$

其中 $m_{(k)}^{(t)}(x_n)$ 表示向量 $m^{(t)}(x_n)$ 的第 k 个坐标。根据错误实现的迭代：

$$m_{(k)}^{(t)}(x_n) = \sum_{i=1}^N W_{ni} m_{(k)}^{(t-1)}(x_i), \quad W_{ni} = \frac{K(x_i, x_n)}{\sum_{j=1}^N K(x_j, x_n)}.$$

于是对该维度有

$$\mathbf{m}_{(k)}^{(t)} = W \mathbf{m}_{(k)}^{(t-1)}, \quad t = 1, 2, \dots, \quad (1)$$

其中矩阵 $W \in \mathbb{R}^{N \times N}$ 定义为

$$W_{ni} = \frac{K(x_i, x_n)}{\sum_{j=1}^N K(x_j, x_n)}, \quad n, i = 1, \dots, N.$$

注意到：

- 对任意固定的行号 n ， $\sum_{i=1}^N W_{ni} = \frac{\sum_{i=1}^N K(x_i, x_n)}{\sum_{j=1}^N K(x_j, x_n)} = 1$ 。因此 W 是一个“行和为 1”的矩阵（行随机矩阵）。
- 由于 RBF 核满足 $K(x_i, x_n) > 0$ （只要两点不无限远），故 $W_{ni} > 0$ ，也就是矩阵 W 每一个元素都严格大于 0。这意味着 W 是“严格正且行随机”的矩阵。

初始时

$$\mathbf{m}_{(k)}^{(0)} = (x_{1,(k)}, x_{2,(k)}, \dots, x_{N,(k)})^T,$$

故下标为 k 的迭代等价于

$$\mathbf{m}_{(k)}^{(t)} = W^t \mathbf{m}_{(k)}^{(0)}.$$

因为 W 是一个“严格正且行和为 1”的矩阵，根据 Perron–Frobenius 定理可知：

- 最大特征值为 $\lambda_1 = 1$ ，唯一且对应的右特征向量 $\mathbf{v}_1$ 与右边不动点有关系。实际上， $\mathbf{v}_1$ 可取为全一向量 $\mathbf{1} = (1, 1, \dots, 1)^T$ ，满足 $W \mathbf{1} = \mathbf{1}$ 。
- 存在唯一的正的、和为 1 的左特征向量 $\boldsymbol{\pi} = (\pi_1, \dots, \pi_N)^T$ ，使得

$$\boldsymbol{\pi}^T W = \boldsymbol{\pi}^T, \quad \sum_{i=1}^N \pi_i = 1, \quad \pi_i > 0.$$

- 由于严格正且行随机，可以将 W 看作一个不可约且非周期的 Markov 链的转移矩阵，所以有

$$\lim_{t \rightarrow \infty} W^t = \begin{pmatrix} \pi_1 & \pi_2 & \cdots & \pi_N \\ \pi_1 & \pi_2 & \cdots & \pi_N \\ \vdots & \vdots & \ddots & \vdots \\ \pi_1 & \pi_2 & \cdots & \pi_N \end{pmatrix},$$

即 W^t 的每一行都收敛到 $\boldsymbol{\pi}^T$ ，收敛速度为指数级。

因此，对于第 k 维坐标，有

$$\mathbf{m}_{(k)}^{(t)} = W^t \mathbf{m}_{(k)}^{(0)} \longrightarrow \begin{pmatrix} \boldsymbol{\pi}^T \mathbf{m}_{(k)}^{(0)} \\ \boldsymbol{\pi}^T \mathbf{m}_{(k)}^{(0)} \\ \vdots \\ \boldsymbol{\pi}^T \mathbf{m}_{(k)}^{(0)} \end{pmatrix}, \quad t \rightarrow \infty.$$

也就是说，对任意 $n = 1, \dots, N$ ，

$$\lim_{t \rightarrow \infty} m_{(k)}^{(t)}(x_n) = \boldsymbol{\pi}^T \mathbf{m}_{(k)}^{(0)} = \sum_{i=1}^N \pi_i m_{(k)}^{(0)}(x_i) = \sum_{i=1}^N \pi_i x_{i,(k)}.$$

对所有 $k = 1, \dots, D$ 维度都成立，说明每个 x_n 的 D 维向量 $m^{(t)}(x_n)$ 都趋向同一个向量。因此“固定核矩阵”的迭代必然收敛，

$$\lim_{t \rightarrow \infty} m^{(t)}(x_n) \text{ 存在，且对所有 } n \text{ 相同。}$$

Answer(2)

我们要求满足

$$\boldsymbol{\pi}^T W = \boldsymbol{\pi}^T, \quad \sum_{i=1}^N \pi_i = 1, \quad \pi_i > 0.$$

记

$$d_i = \sum_{j=1}^N K(x_i, x_j), \quad i = 1, \dots, N.$$

考虑猜测

$$\pi_i = \frac{d_i}{\sum_{\ell=1}^N d_\ell}, \quad i = 1, \dots, N.$$

验证如下：对于任意 i ,

$$\begin{aligned} \sum_{n=1}^N \pi_n W_{ni} &= \sum_{n=1}^N \frac{d_n}{\sum_{\ell=1}^N d_\ell} \cdot \frac{K(x_i, x_n)}{\sum_{j=1}^N K(x_j, x_n)} \\ &= \frac{1}{\sum_{\ell=1}^N d_\ell} \sum_{n=1}^N \frac{d_n}{d_n} K(x_i, x_n) = \frac{1}{\sum_{\ell=1}^N d_\ell} \sum_{n=1}^N K(x_i, x_n) = \frac{d_i}{\sum_{\ell=1}^N d_\ell} = \pi_i. \end{aligned}$$

由于 $K(x_i, x_n) = K(x_n, x_i)$, 因此上式成立。可见

$$\boldsymbol{\pi} = (\pi_1, \dots, \pi_N)^T = \left(\frac{d_1}{\sum_{\ell} d_\ell}, \frac{d_2}{\sum_{\ell} d_\ell}, \dots, \frac{d_N}{\sum_{\ell} d_\ell} \right)^T, \quad d_i = \sum_{j=1}^N K(x_i, x_j),$$

是 W 唯一的正的、和为 1 的左特征向量。

如上所述，对任意维度 k :

$$\lim_{t \rightarrow \infty} m_{(k)}^{(t)}(x_n) = \sum_{i=1}^N \pi_i x_{i,(k)}.$$

于是对完整的 D 维向量，有

$$\lim_{t \rightarrow \infty} m^{(t)}(x_n) = \sum_{i=1}^N \pi_i x_i, \quad \text{对所有 } n = 1, \dots, N \text{ 都相同,}$$

其中

$$\pi_i = \frac{\sum_{j=1}^N K(x_i, x_j)}{\sum_{\ell=1}^N \sum_{j=1}^N K(x_\ell, x_j)}.$$

即所有 $m^{(t)}(x_n)$ 最终聚到同一个常数向量

$$\boldsymbol{\mu} = \sum_{i=1}^N \pi_i x_i,$$

这个 $\boldsymbol{\mu}$ 刚好是对原始样本 $\{x_i\}$ 按“与其它点的 RBF 相似度之和”加权平均所得的向量。

简答题

Answer(1)

生成型模型：高斯混合模型（GMM）

- **模型设计：** GMM 通过建模每个类别下的特征分布 $P(x|y)$ 来估计后验概率 $P(y|x)$ 。假设每一类的特征分布为高斯混合模型：

$$P(x|y=c) = \sum_{k=1}^K w_k^{(c)} \mathcal{N}(x|\mu_k^{(c)}, \Sigma_k^{(c)})$$

- 学习方法：使用 EM 算法估计参数 $\mu_k^{(c)}$ 、 $\mu_k^{(c)}$ 和 $\Sigma_k^{(c)}$ ，通过贝叶斯公式计算 $P(y|x)$ 进行分类。
- 公平性改进方法：
 - 可在建模时剔除“性别”和“年龄”等敏感属性；
 - 对生成分布进行正则化约束，使其在不同性别和年龄组上的分布尽量保持一致；

判别型模型：逻辑回归（Logistic Regression）

- 模型设计：逻辑回归直接建模后验概率：

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

其中 x 是输入特征，包括“性别、年龄、毕业学校、工作经历”。

- 学习方法：最小化交叉熵损失函数，通过梯度下降等方法求解模型参数 w, b 。
- 公平性改进：
 - 可在训练时移除“性别”和“年龄”特征；
 - 引入公平性正则项，例如对不同性别组的输出概率差异进行惩罚：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{fairness}}$$

Answer(2)

在实现公平性方面，生成型模型比判别型模型更难实现公平性，因为

生成模型需要建模 $P(x|y)$ ，难以从模型结构中直接剥离敏感特征的影响，一旦数据中存在结构性偏差，模型往往会继承并放大这种偏差。

判别模型则直接建模 $P(y|x)$ ，易于通过去除敏感特征、加入公平正则项或使用对抗训练等方式对模型进行控制和调整。

因此，从公平性角度出发，判别型模型更容易进行调控与干预。